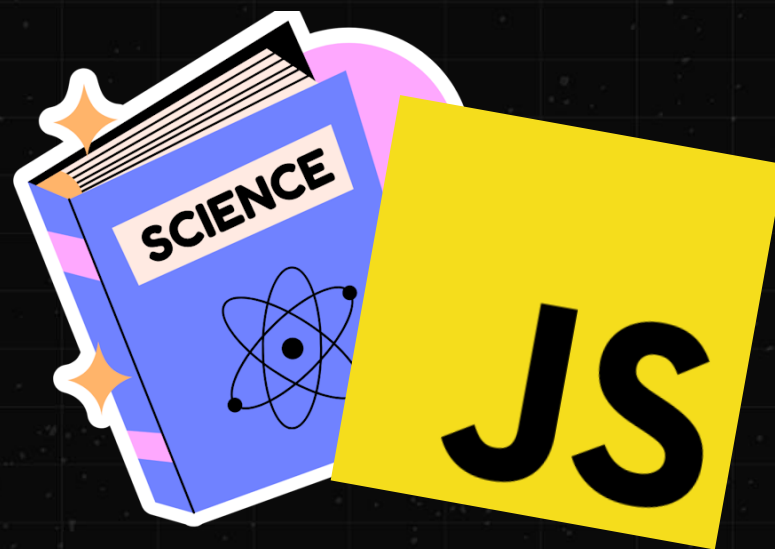


ЛЕКЦИЯ #16

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Познакомиться с семейством структур данных — деревьями поиска
- * Разобрать отличия между бинарными и многопутевыми деревьями
- * Познакомиться с понятием балансировки дерева
- * Обсудить некоторые реализации и сравнить их с хэш-таблицами

ВСПОМНИМ ПРО ДЕРЕВЬЯ

- * Это подвид графа, в котором все вершины связаны (связный граф)
- * Нет циклов (ациклический граф)
- * Любые две вершины соединяются ровно одним путём
- * Количество рёбер строго равно $V - 1$

«КОРНЕВОЕ» ДЕРЕВО

- * Ориентированное дерево
- * Есть ровно один узел, в который не входит ни одна дуга – корень дерева
- * Из корня в каждую вершину ведёт ровно один путь
- * Узлы, у которых нет потомков, называются листьями



КСТАТИ

- * Дерево на прошлом изображении обладало ещё двумя свойствами
- * У каждой вершины **не может быть более двух потомков** — такое дерево называется бинарным
- * **Значения в узлах отсортированы** (упорядоченное дерево): левый потомок меньше родителя, а правый — больше
- * Такое дерево называется **бинарным деревом поиска**

ЧТО НАМ
ЭТО ДАЛО?

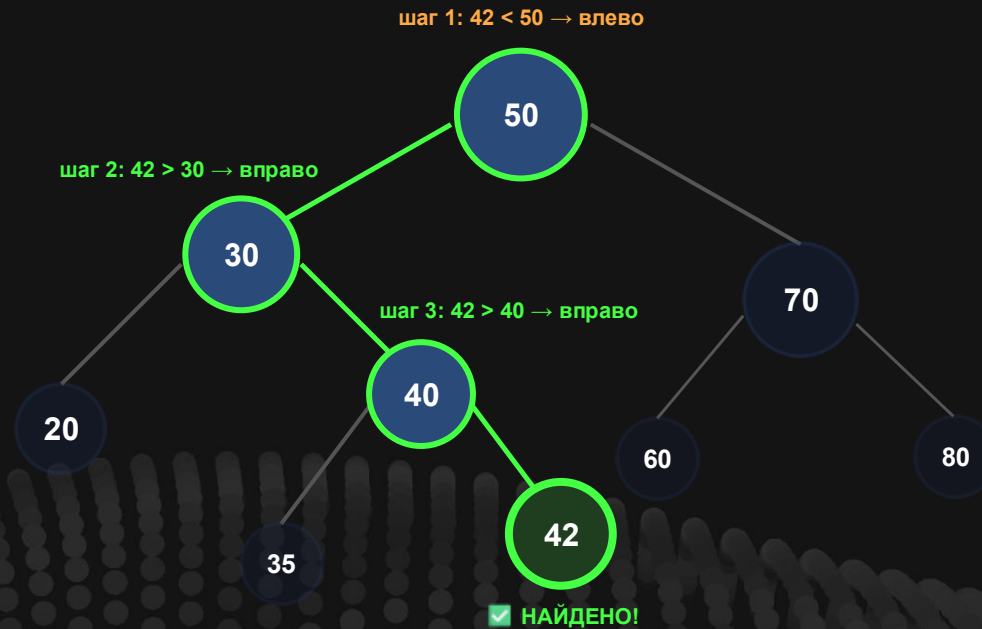


ЭФФЕКТИВНЫЙ ПОИСК, ВСТАВКА И УДАЛЕНИЕ

- * Чтобы найти некоторое значение, мы **всегда начинаем путь с корня**
- * **На каждом уровне мы делаем шаг «влево» или «вправо»**, так как дерево упорядочено
- * Фактически это **совпадает с алгоритмом бинарного поиска**
- * Операции вставки и удаления **не требуют пересортировки или сдвига элементов** — мы просто меняем ссылки
- * Сложность этих операций получается $O(\log n)$

Поиск в бинарном дереве поиска (BST)

Поиск значения 42 — пройденный путь от корня



Пошаговый процесс поиска

1. Сравниваем 42 с корнем (50). $42 < 50 \rightarrow$ идём в левое поддерево
2. Сравниваем 42 с узлом 30. $42 > 30 \rightarrow$ идём в правое поддерево
3. Сравниваем 42 с узлом 40. $42 > 40 \rightarrow$ идём вправо и находим значение 42!

пройденный путь найденный узел не посещённые узлы путь поиска

В бинарном дереве поиска каждый шаг отсекает половину поддерева, обеспечивая $O(\log n)$ в среднем

НО ТУТ ЕСТЬ НЮАНС

- * Алгоритм будет эффективен лишь тогда, когда высоты всех ветвей **будут примерно одинаковы**
- * Такое дерево **называется сбалансированным**
- * Ветвь — это **путь от корня до какого-либо листа** (конечного узла)
- * Высота узла — это **максимальная длина пути от этого узла до листа** в его поддереве
- * Только вот операции вставки и удаления нарушают баланс

Нарушение баланса в бинарном дереве поиска

Добавление узлов в отсортированном порядке приводит к вырождению в связный список

Шаг 1: добавляем 10



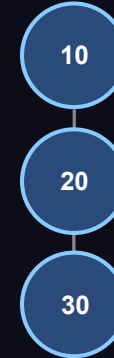
корень
высота = 1

Шаг 2: добавляем 20



$20 > 10 \rightarrow$ вправо
высота = 2

Шаг 3: добавляем 30



$30 > 20 \rightarrow$ вправо
высота = 3

Сравнение сбалансированного и вырожденного дерева

✓ Сбалансированное дерево



высота = 2
поиск: $O(\log 3) \approx 2$ шага

✗ Вырожденное дерево



высота = 3
поиск: $O(3) \approx 3$ шага

Вывод: добавление узлов в отсортированном порядке $\rightarrow$ вырождение в связный список $\rightarrow$ деградация поиска до $O(n)$

Для сохранения $O(\log n)$ необходимы самобалансирующиеся деревья: AVL, красно-чёрное (Red-Black), B-деревья

РЕФЛЕКСИРУЕМ

- * Дерево поиска **эффективно** лишь тогда, когда оно **сбалансировано**
- * Операции **вставки** и **удаления** **нарушают баланс**
- * Дерево **может выродиться** в **связный список** (линейный граф)
- * Различные виды деревьев поиска используют **особые алгоритмы вставки и удаления**, чтобы дерево всегда оставалось сбалансированным

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

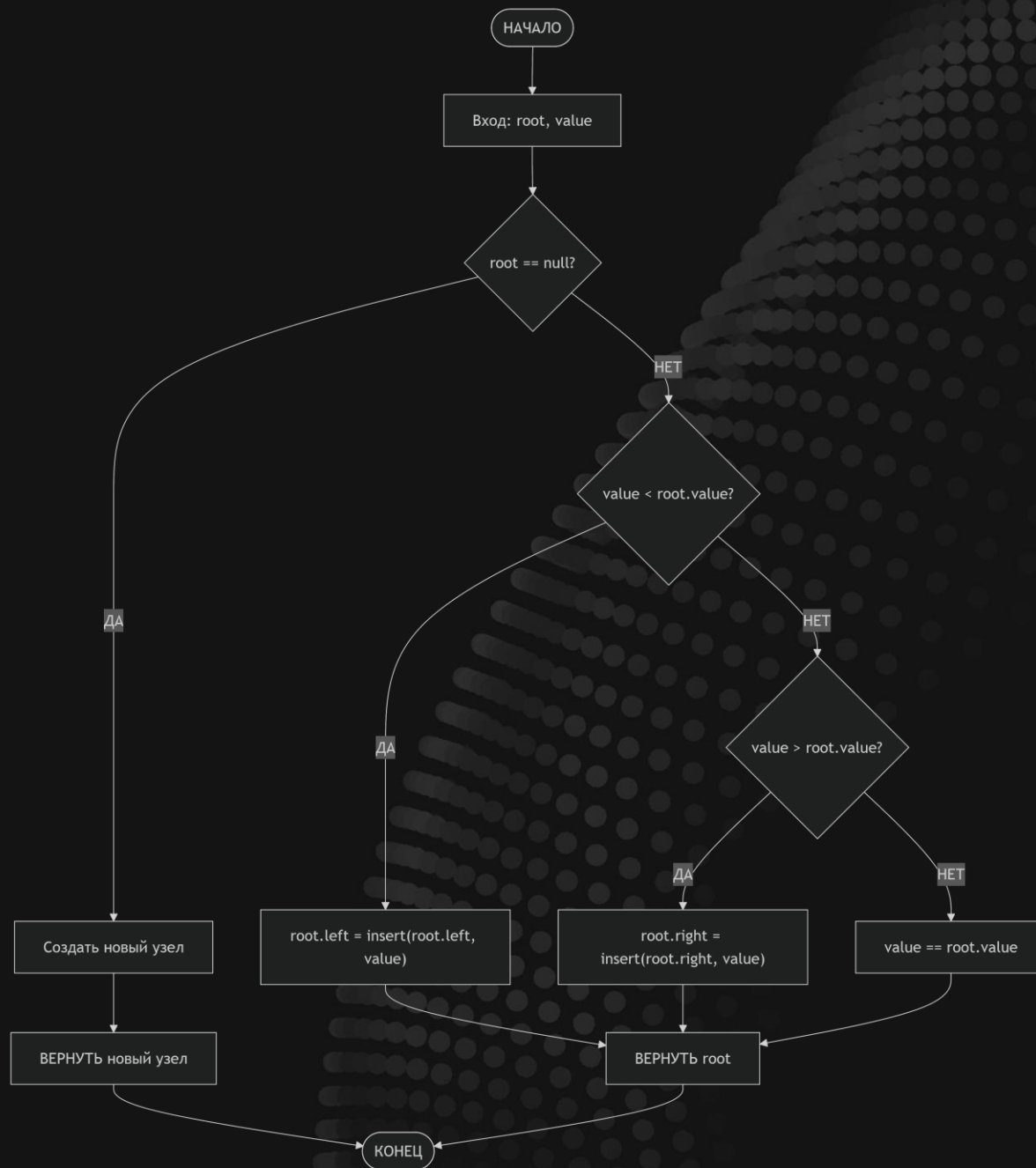
ОТСТАВИТЬ ПАНИКУ

- ✱ Мы не будем сегодня детально разбирать все существующие деревья поиска
- ✱ Вместо этого мы выделим **основные принципы и те преимущества**, которые дают нам деревья поиска

А КАК ВООБЩЕ ДЕЛАЕТСЯ ВСТАВКА И УДАЛЕНИЕ В БИНАРНОЕ ДЕРЕВО ПОИСКА?



ВСТАВКА



УДАЛЕНИЕ

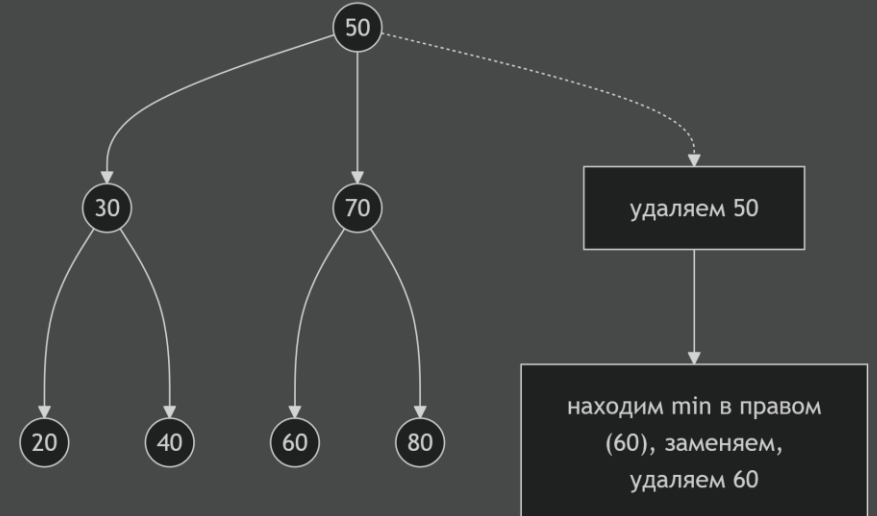
Случай 1: Лист



Случай 2: Один потомок



Случай 3: Два потомка



РЕФЛЕКСИРУЕМ

- * Вставка и удаление совмещают в себе **алгоритм поиска и операцию изменения**
- * Удаление – более сложная операция, но на её примере мы видим **дополнительные свойства такого дерева**
- * Мы можем **легко найти экстремумы** – минимальное и максимальное значение
- * Ключи отсортированы, а значит, можно находить не только конкретный ключ, **но и диапазоны значений**

ЧТО ЗНАЧИТ ДИАПАЗОНЫ?



НАПРИМЕР

- * Если ключом является дата, то можно запросить **множество всех значений в заданном временном отрезке**
- * При этом операция **будет выполняться за логарифмическое время** (если дерево сбалансированное)
- * А вот в хэш-таблице такая операция потребовала бы **полного перебора всей коллекции**

КАКИЕ ЕСТЬ ЕЩЕ
ПЛЮСЫ ПО СРАВНЕНИЮ
С ХЭШ-ТАБЛИЦЕЙ?



ПРЕДСКАЗУЕМОСТЬ

- * В сбалансированном дереве операции выполняются с **гарантированной алгоритмической сложностью**
- * Нет коллизий и перехэширования
- * Память выделяется более плавно
- * Однако в сценарии чтения по фиксированному ключу хэш-таблицы **на практике всё равно быстрее**

КОГДА НУЖНЫ ДЕРЕВЬЯ ПОИСКА

- * **Упорядоченный обход данных** — например, вывод записей в алфавитном порядке
- * **Поиск ближайших соседей** — найти элемент, наиболее близкий к заданному значению (например, ближайшую дату, цену или координату)
- * **Поиск по диапазону** — найти все элементы между A и B (например, товары от 1000 до 5000 рублей или события за прошлую неделю)
- * **Получение следующего / предыдущего элемента**
- * **Префиксный поиск (автодополнение)**
- * **Многомерные данные** — например, поиск ближайшей точки на карте

КОГДА ПОИСК ТОЛЬКО ПО КЛЮЧУ

- ✱ Если нужен только точный поиск по ключу (без диапазонов, экстремумов и упорядоченного обхода), то **хэш-таблица будет эффективнее**
- ✱ Она даёт **амортизированное $O(1)$** против $O(\log n)$ у сбалансированного дерева

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ЗАПОМНИТЕ СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

- * Когда вы столкнетесь с таким сценарием, то **сразу будет понятно, куда копать**
- * **Вместо того, чтобы городить непонятные костыли** на хэш-таблицах и массивах

КАК ПРЕДСТАВИТЬ ДЕРЕВО В ПАМЯТИ?



ДЕРЕВО – ЭТО ГРАФ

- * Мы можем использовать **любой** из способов представления графа
- * Но **матричные представления** будут **слишком избыточны** для дерева (разреженный граф)
- * Нам нужно **уметь «растить» дерево** – динамически добавлять узлы
- * Значит, нам подойдёт **структура смежности**
- * А чтобы представить бинарное дерево, просто **будем хранить две ссылки – left и right**

```
interface BinNodeChildren<T> {
    left?: BinNode<T>;
    right?: BinNode<T>;
}

class BinNode<T> {
    value: T;

    left: BinNode<T> | null;
    right: BinNode<T> | null;

    constructor(value: T, { left, right }: BinNodeChildren<T> = {}) {
        this.value = value;
        this.left = left ?? null;
        this.right = right ?? null;
    }
}

class BinaryTree<T> {
    root: BinNode<T>;

    constructor(value: T, { left, right }: BinNodeChildren<T> = {}) {
        this.root = new BinNode(value, { left, right });
    }
}
```

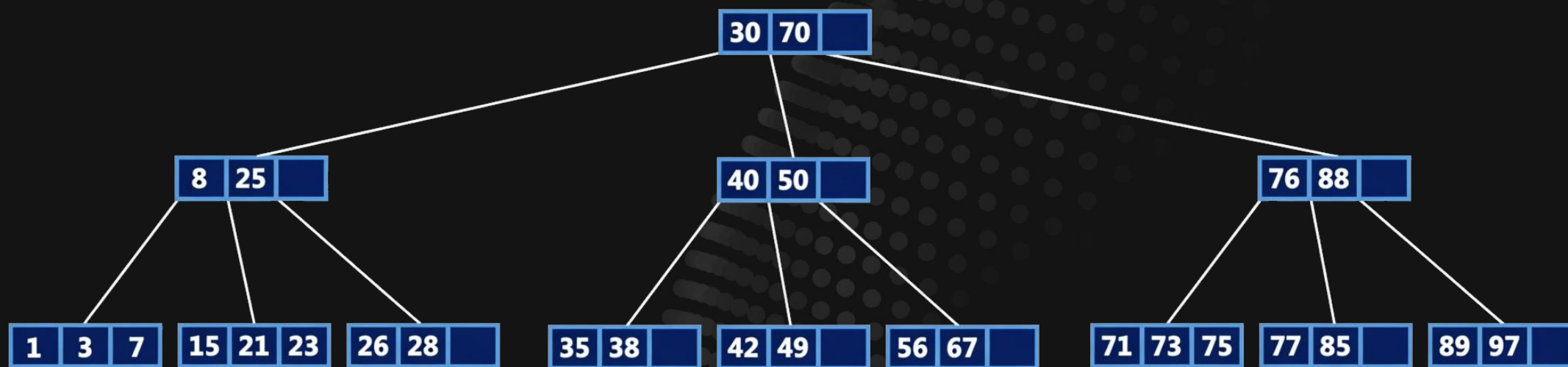
А КАК ТУТ С
ЛОКАЛЬНОСТЬЮ?



НЕ ОЧЕНЬ

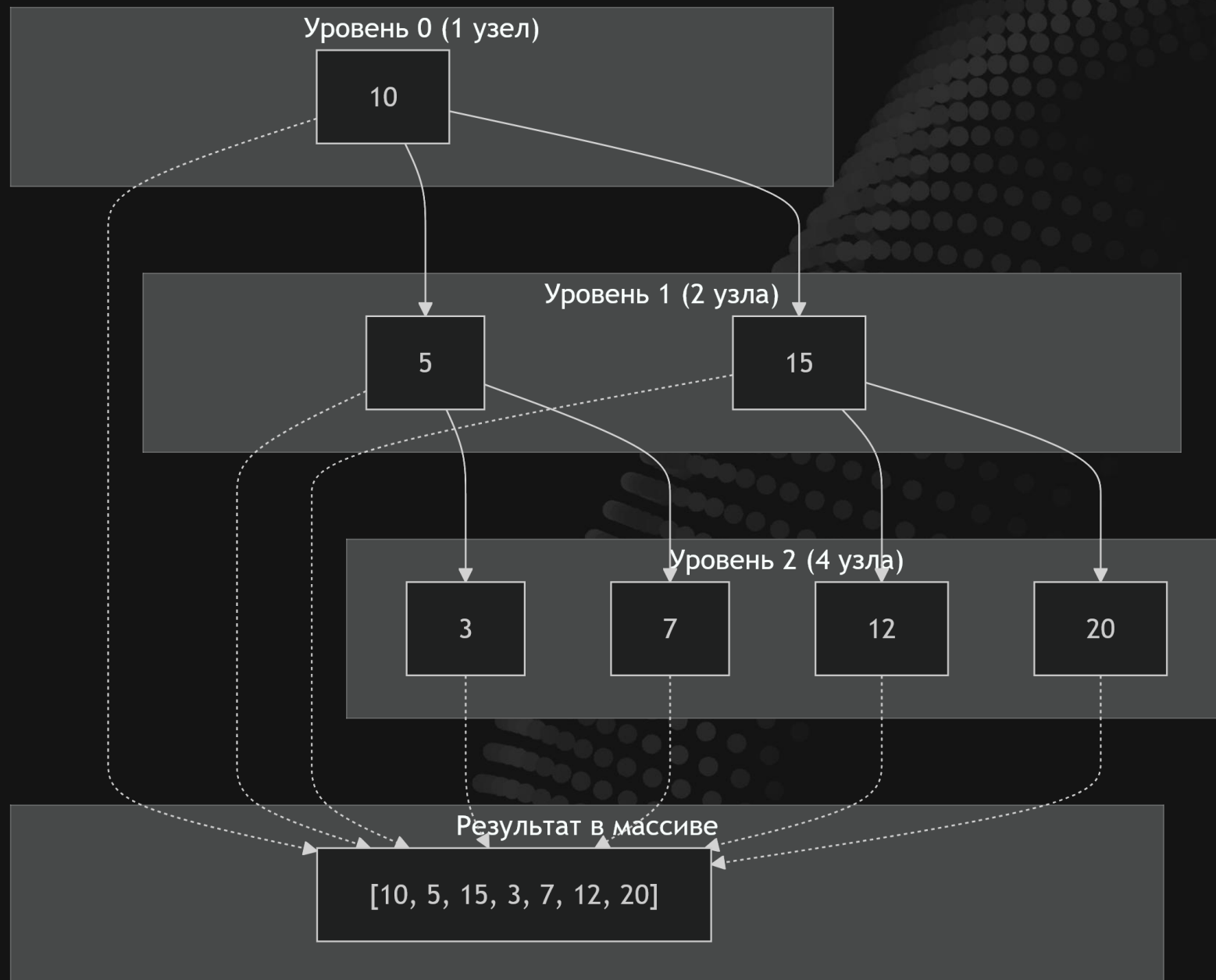
- * По сути, у нас **тот же связный список, но с другими контрактами**
- * Значения сильно **разбросаны в памяти**, что плохо для кэша
- * Можно исправить ситуацию, перейдя на **хранение массива значений в узле**
- * **Между соседними ключами хранятся ссылки на поддеревья**
- * Такое дерево **называется многопутевым**, так как у каждого узла может быть больше двух потомков

НАПРИМЕР



ЕСТЬ ЕЩЕ СПОСОБЫ

- * Любое бинарное дерево можно «вложить» в массив
- * При этом **индексы левого и правого потомков вычисляются** по формулам:
 $2 * i + 1$ и $2 * i + 2$
- * Доступ к родительскому узлу можно получить по формуле: $\text{Math.floor}((i - 1) / 2)$
- * Дополнительный плюс такого подхода — **не нужно хранить отдельно ссылки на узлы**
- * Для динамического роста можно использовать обычный вектор



И ЕЩЕ СПОСОБ

- ✱ Любое дерево или связный список можно вложить в массив — роль индексов выполняют ячейки массива
- ✱ Такой подход позволяет повысить локальность и контролировать выделение памяти
- ✱ Это особенно полезно при работе с типизированными массивами

```

class FlatTree {
  #nodes = [];
  #rootIndex = -1;

  addNode(value) {
    const node = {
      value: value,
      children: [],
      parentIndex: -1
    };

    this.#nodes.push(node);
    return this.#nodes.length - 1;
  }

  addChild(parentIndex, childIndex) {
    if (parentIndex < 0 || parentIndex >= this.#nodes.length) return;
    if (childIndex < 0 || childIndex >= this.#nodes.length) return;

    this.#nodes[parentIndex].children.push(childIndex);
    this.#nodes[childIndex].parentIndex = parentIndex;
  }

  setRoot(index) {
    if (index < 0 || index >= this.#nodes.length) return;
    this.#rootIndex = index;
  }

  getValue(index) {
    if (index < 0 || index >= this.#nodes.length) return null;
    return this.#nodes[index].value;
  }
}

```

```

getChildren(index) {
  if (index < 0 || index >= this.#nodes.length) return [];
  return [...this.#nodes[index].children];
}

getParent(index) {
  if (index < 0 || index >= this.#nodes.length) return -1;
  return this.#nodes[index].parentIndex;
}

traverse(cb, nodeIndex = this.#rootIndex) {
  if (nodeIndex === -1) return;

  const node = this.#nodes[nodeIndex];
  cb(node.value, nodeIndex);

  for (const childIndex of node.children) {
    this.traverse(cb, childIndex);
  }
}

```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВСЕ НЕ ТАК СТРАШНО

- * Вся память – **это большой массив**
- * Поэтому естественно, что мы **можем взять массив поменьше** и работать с ним как с памятью
- * Более того, на практике так очень часто и делают

А ЧТО ТАМ С
БАЛАНСИРОВКОЙ?



СБАЛАНСИРОВАННЫЕ ДЕРЕВЬЯ В ОБЩЕМ

- * Это упорядоченные бинарные или многопутевые деревья
- * Операции вставки и удаления выполняют перебалансировку дерева согласно алгоритму конкретной структуры данных
- * Обычно эти операции также занимают $O(\log n)$
- * Таким образом, такое дерево всегда остаётся сбалансированным

АВЛ-ДЕРЕВО

- * AVL – аббревиатура, образованная первыми буквами создателей (советских учёных) Адельсона-Вельского Георгия Максимовича и Ландиса Евгения Михайловича
- * Сбалансированное **бинарное дерево поиска**
- * Балансировка происходит **при вставке и удалении**
- * Визуализация алгоритма

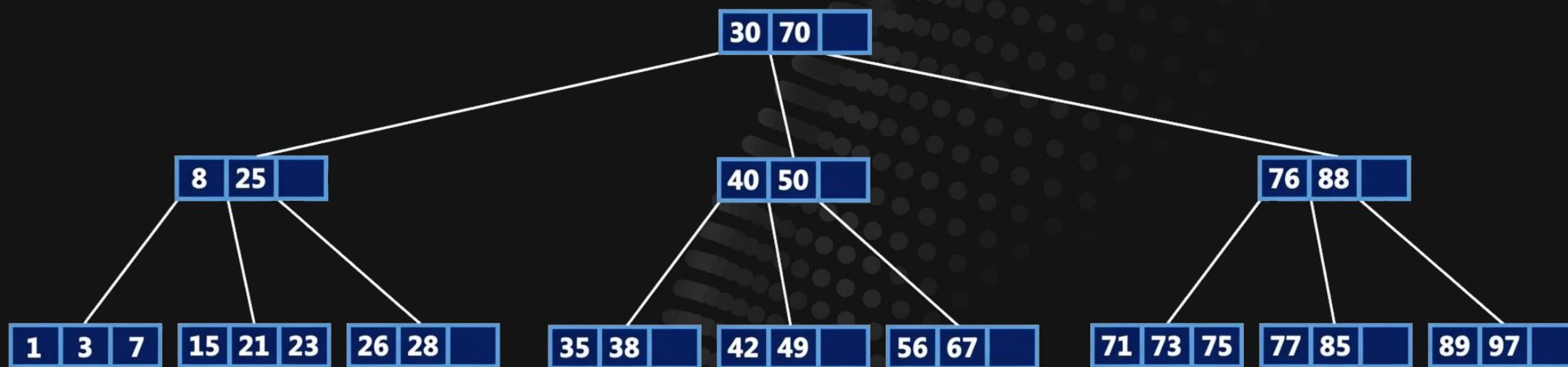
КРАСНО-ЧЕРНОЕ ДЕРЕВО

- * Сбалансированное бинарное дерево поиска
- * Все узлы имеют цвет (красный или чёрный)
- * Балансировка происходит при вставке и удалении
- * [Визуализация алгоритма](#)

В-ДЕРЕВО

- * Сбалансированное **многопутевое** дерево поиска
- * В узлах хранится **не одно значение, а массив** (отсортированный по ключу)
- * Максимальное **количество детей в узле можно настраивать** (порядок дерева)
- * Балансировка происходит **при вставке и удалении** – узлы расщепляются или сливаются
- * [Визуализация алгоритма](#)

НАПРИМЕР



В-ДЕРЕВО VS БИНАРНЫЕ ДЕРЕВЬЯ

- * В-дерево по умолчанию **имеет хорошую локальность данных** (узел – непрерывный блок памяти)
- * Управляя размером узла, можно **оптимизировать работу кэша и скорость поиска**
- * **В-дерево требует больше памяти**, особенно в сравнении с бинарным деревом на массиве (из-за хранения массивов ключей и ссылок)

В+-ДЕРЕВО

- * **Модификация В-дерева**, популярная для организации файловых систем и баз данных
- * В В+ дереве **данные сохраняются только в листовых узлах**
- * **Внутренние узлы содержат только ключи** (для навигации) и ссылки на детей, но не содержат данных
- * Листовые узлы представляют собой связный список
- * [Визуализация алгоритма](#)

В ЧЕМ ТУТ
СМЫСЛ?



ЗАГРУЗКА ПО ТРЕБОВАНИЮ

- * При работе с B+-деревом можно держать в памяти только рабочие узлы (индекс), а **сами данные загружать по требованию** (lazy loading)
- * Это позволяет работать с огромными объёмами данных, которые **целиком не помещаются в оперативную память**
- * Например, СУБД или файловая система держат в памяти только индекс, а страницы с данными подгружаются с диска при необходимости
- * Или для организации офлайн-работы – загружать лишь некоторые словари (например, в мобильных приложениях)

В-ДЕРЕВО И В+-ДЕРЕВО

- * Выбор между В-деревом и В+ деревом обуславливается тем, **нужна ли вам ленивая подгрузка данных**
- * В+ дерево лучше подходит для сценариев, когда данные не помещаются в память и нужно подгружать их по требованию (диск, сеть)
- * В-дерево будет быстрее для сценария, когда все данные находятся в оперативной памяти (нет необходимости в lazy loading)

second monitor

\$0.00 (0%)

50.00

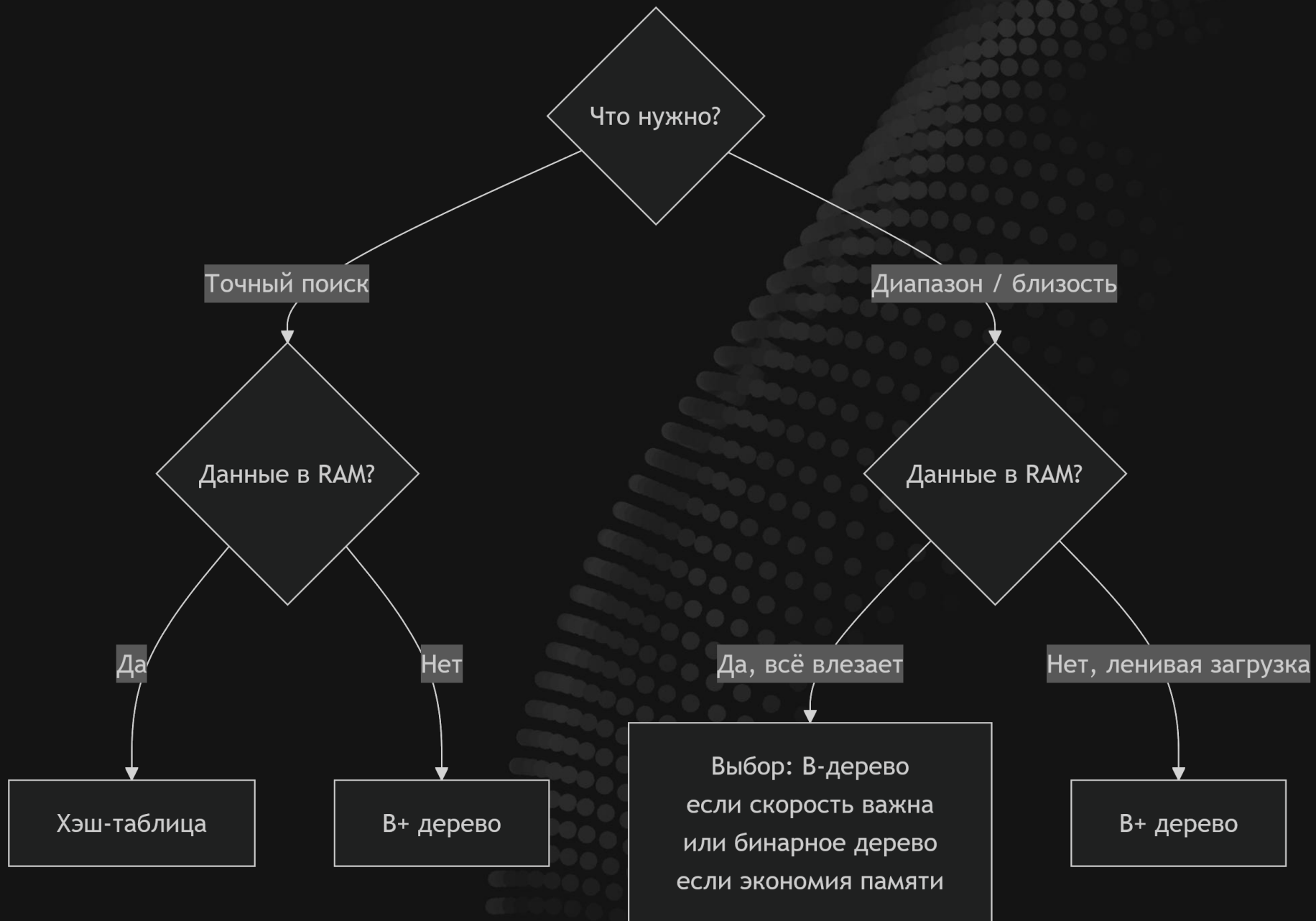
224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОНИМАЮ ВАШИ ЧУВСТВА

- ✱ Но наша задача — **понять «зачем и когда это нужно»**
- ✱ Если вы понимаете это, то сможете выбрать нужную библиотеку или реализовать структуру самостоятельно с помощью LLM
- ✱ **Понимание принципов важнее** запоминания конкретных реализаций



R-ДЕРЕВО

- * Еще одна разновидность дерева, похожая на B+, но в качестве ключей выступают **геометрические объекты** (в простой реализации – прямоугольники)
- * Позволяет **эффективно искать вхождения и пересечения одних объектов с другими**
- * Например, расчёт коллизий в компьютерных играх
- * Есть похожие деревья для точек (K-D дерево) и другие варианты

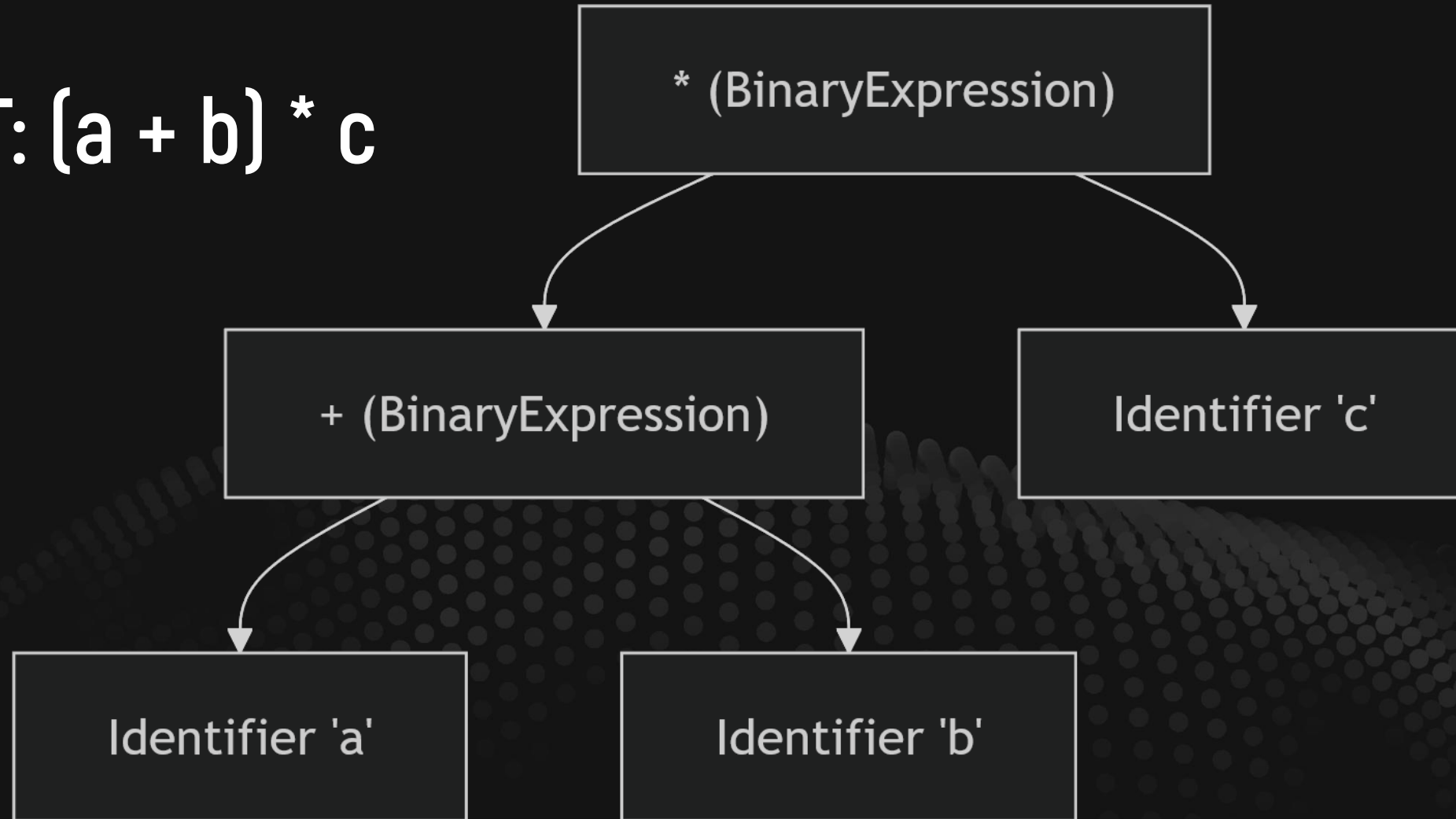
РЕФЛЕКСИРУЕМ

- * Разбирая деревья поиска, можно прийти к выводу, что их можно разделить **не только на бинарные и многопутевые**
- * Но и по тому, **что является ключом в таком дереве**
- * Существуют универсальные деревья, которые работают **с любым «сравнимым» ключом** (например, числа, строки, даты)
- * А есть специализированные деревья, которые **вводят строгий контракт** на тип ключа и операцию сравнения
- * Например, R-дерево работает с геометрическими объектами

НЕ ТОЛЬКО ПОИСК

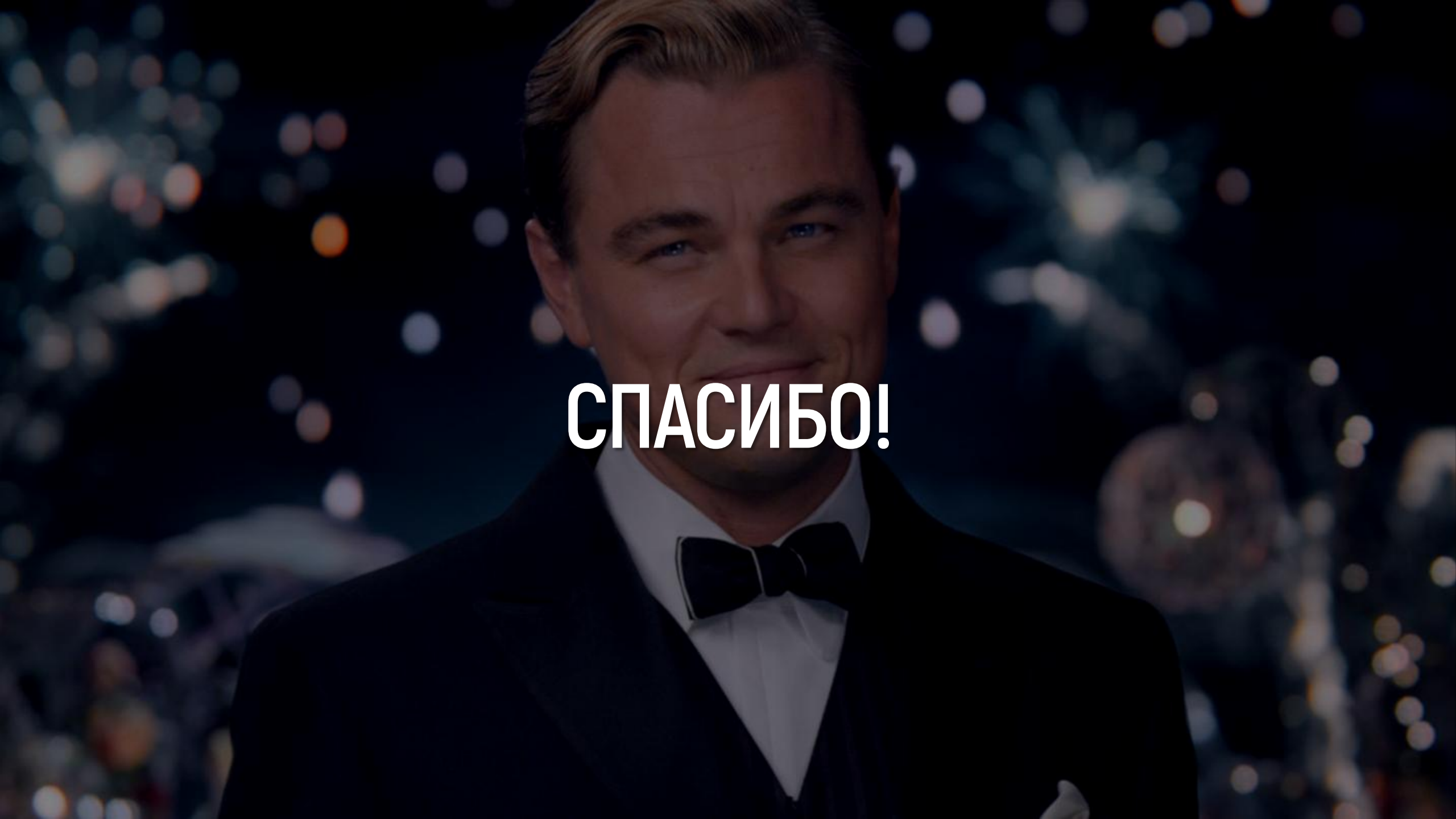
- * Деревья используются **не только для операций поиска**, но и просто для представления отношений между объектами
- * Например, DOM-дерево в браузере описывает структуру веб-страницы
- * Или AST-дерево (абстрактное синтаксическое дерево) **для представления структуры кода в компиляторах и парсерах**

AST: $(a + b) * c$



ПОДВЕДЕМ ИТОГИ

- * Деревья являются одной из важнейших структур данных в программировании
- * Главное отличие от хэш-таблицы: деревья работают с **упорядоченными данными**
- * Это позволяет им делать операции, которые **хэш-таблица принципиально не поддерживает**: поиск по диапазону, поиск экстремумов
- * При этом деревья остаются **эффективными для операций вставки, удаления и точного поиска** – $O(\log n)$
- * В+ деревья позволяют работать с данными, которые не помещаются в память (диск, сеть), загружая только индекс

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!